

NUS-ISS Intelligent Reasoning Systems

Practice Module

FleetCare AI

A Hybrid Temporal Reasoning System for Predictive Maintenance of Heavy-Duty Trucks

GROUP MEMBERS

Zhang Tianqin	A0356380B
Zhang Yuyue	A0305487X
Ma Jialing	A0305478X
Gong Yucheng	A0355325E

23 October 2026

Table of Contents

1 Introduction . . . 3
2 Project Background . . . 3
2.1 Predictive-maintenance problem . . . 3
2.2 Commercial and decision-support context . . . 4
2.3 Final system positioning . . . 4
3 Project Scope . . . 5
3.1 Implemented capabilities . . . 5
3.2 Scope boundaries and limitations . . . 6
4 Data Collection and Preparation . . . 6
4.1 Data Source and Characteristics . . . 6
4.2 Target construction and censoring . . . 8
4.3 Leakage control and endpoint alignment . . . 9
4.4 Final feature representation . . . 9
5 System Design . . . 9
5.1 Overall Architecture . . . 9
5.2 Database Design . . . 10
5.3 Predictive Model . . . 11
5.4 Rule engine . . . 12
5.5 Inference, persistence, and explainability . . . 13
5.6 RAG-Based Assistant . . . 13
5.7 Application Layer . . . 14
6 Predictive Modelling Experiments . . . 19
6.1 Experimental protocol and evaluation criteria . . . 20
6.2 Baseline model families . . . 20
6.3 Feature-representation experiments . . . 20
6.4 Class imbalance, sampling, and weighting . . . 21
6.5 Calibration and probability experiments . . . 21
6.6 Specialist C4 ranking and ensemble experiments . . . 22
6.7 Temporal degradation and sequence experiments . . . 22
6.8 Endpoint alignment and final tree tuning . . . 23
6.9 Key findings and model-selection rationale . . . 24
7 RAG Evaluation . . . 24
7.1 Evaluation Setup . . . 24
7.2 Evaluation Protocol . . . 24
7.3 Results and Analysis . . . 25
8 Conclusion . . . 26
9 Future Work . . . 26
10 Appendix . . . 27
10.1 Installation and User Guide . . . 27
10.2 Mapped System Functionalities . . . 27
10.3 Proposal Report . . . 29

1 Introduction

FleetCare AI is a hybrid intelligent reasoning system for prioritising predictive-maintenance inspections in heavy-duty vehicle fleets. It uses the public SCANIA Component X dataset to learn from longitudinal operational readouts and vehicle specifications, rank active vehicles by near-term maintenance risk, and convert predictive evidence into traceable operational actions. The completed system integrates a versioned machine-learning package, a temporal rule engine, PostgreSQL persistence, SHAP-based explanations, a FastAPI backend, a React fleet-management dashboard, and a retrieval-augmented assistant.

The project addresses a practical limitation of reactive and fixed-interval maintenance. Heavy-duty vehicles operate under different specifications, exposure levels, loads, and historical conditions; vehicles of similar age may therefore require different levels of attention. Data-driven modelling can help identify vehicles whose recent operating patterns resemble those observed close to a maintenance event. However, a prediction alone does not determine how limited maintenance resources should be allocated. Fleet managers must also decide which vehicles should enter the inspection queue, how urgently they should be handled, whether their historical evidence is sufficient and stable, and why a particular action has been recommended.

FleetCare AI addresses these requirements through a model-first, human-in-the-loop architecture. A specialist C4 ranking ensemble combines XGBoost and LightGBM to prioritise vehicles approaching the most urgent 0–6-unit maintenance stage. A separate five-class XGBoost model provides the displayed C0–C4 repair-proximity state and coherent Risk6, Risk12, Risk24, and Risk48 probabilities. An ordered temporal rule engine then combines the fleet-relative ranking with recent persistence, deterioration, model disagreement, and history sufficiency. It translates this evidence into actions ranging from `CONTINUE` to `PRIORITY_INSPECT`, with separate routes for unstable predictions and insufficient data.

Each assessment is stored in PostgreSQL together with its model outputs, matched rule, reason, temporal evidence, and strongest SHAP contributions. This separation preserves the distinction between prediction and operational policy: the model estimates and ranks risk, while the rule engine determines how that evidence should be handled. The dashboard exposes both fleet-level priorities and vehicle-level evidence, allowing users to inspect current risk, historical changes, recommended actions, and the reasoning behind each recommendation. The system supports professional maintenance judgement rather than diagnosing a physical fault or autonomously authorising maintenance.

The retrieval-augmented assistant provides two controlled evidence paths. Vehicle-specific questions retrieve stored assessments, recent risk history, rule traces, actions, and SHAP factors from PostgreSQL. Maintenance and policy questions retrieve citation-bearing passages from a curated knowledge base. Function Calling selects the appropriate retrieval tool, while the assistant remains an explanation interface: it does not recalculate predictions or modify the action assigned by the rule engine.

The completed project has five objectives:

- rank active vehicles by near-term C4 risk under a fixed inspection budget;
- retain five repair-proximity states and coherent Risk6, Risk12, Risk24, and Risk48 outputs for vehicle-level communication;
- transform model and temporal evidence into transparent operational actions, including explicit handling of unstable predictions and insufficient history;
- preserve an auditable evidence chain containing predictions, temporal indicators, matched rules, reasons, and SHAP contributions; and
- present fleet priorities, vehicle histories, explanation evidence, and grounded assistant responses through an integrated decision-support interface.

FleetCare AI therefore connects predictive modelling, temporal reasoning, explainability, structured data management, and retrieval-augmented interaction within one decision-support workflow. The resulting assessments are designed to be prioritised, traceable, and reviewable by human decision makers.

2 Project Background

2.1 Predictive-maintenance problem

The SCANIA Component X benchmark represents maintenance proximity using five ordered operational-time intervals. These intervals are measured in benchmark time units rather than calendar days and do

Table 1: Repair-proximity classes used by FleetCare AI.

Class	Time to recorded maintenance	System interpretation
C0	More than 48 units	No event within the 48-unit horizon
C1	24–48 units	Early repair proximity
C2	12–24 units	Elevated repair proximity
C3	6–12 units	Near-term repair proximity
C4	0–6 units	Most imminent repair-proximity stage

not identify a specific mechanical failure mechanism. The prediction task therefore concerns proximity to a recorded maintenance event, rather than diagnosis of a named physical component.

The modelling problem is difficult for several reasons. First, C0 dominates the dataset, while C4 is rare. A model can therefore achieve high conventional accuracy while failing to identify the vehicles that matter most under a limited inspection budget. Second, vehicles have histories of different lengths, and their readouts are irregularly spaced in operational time. Third, most operational variables are cumulative, meaning that raw values are strongly influenced by accumulated vehicle exposure. Fourth, vehicles without an observed maintenance event are right-censored: the absence of a recorded event does not imply that the vehicle has infinite remaining life.

These characteristics motivated vehicle-level data separation, past-only temporal features, exposure-normalised feature engineering, careful treatment of censored observations, and evaluation based on C4 recovery within a fixed fleet-review capacity. They also motivated the separation of predictive assessment from operational decision making. A model can estimate and rank risk, but an additional reasoning layer is required to determine how the available evidence should be handled.

2.2 Commercial and decision-support context

The commercial fleet-management market increasingly uses connected-vehicle data to reduce unplanned downtime, improve vehicle availability, and support proactive maintenance planning [1]. Major platforms illustrate this direction from complementary perspectives. Scania Digital Services applies connected-vehicle information to monitoring and service planning [7]; Geotab combines fleet analytics, diagnostics, predictive-maintenance insights, and maintenance workflows [3]; and Samsara links real-time diagnostics and fault monitoring with scheduling and operational processes [6]. IBM Maximo demonstrates the wider industrial adoption of predictive asset-health and maintenance-management capabilities beyond vehicle fleets [4].

Together, these platforms demonstrate established commercial demand for converting operational data into timely and actionable maintenance decisions. FleetCare AI addresses a specific problem within this market: how predictive evidence can be translated into a transparent, inspectable, and capacity-aware maintenance response.

Machine-learning methods are well established in predictive-maintenance research [2], including automotive applications that use vehicle operating data for data-driven maintenance assessment [8]. However, predictive performance alone does not provide a complete maintenance workflow. Operational users must also understand why a vehicle has been prioritised, whether its recent history supports the assessment, and what action should follow from the available evidence.

FleetCare AI addresses this gap by connecting predictive ranking with explicit temporal reasoning and evidence delivery. SHAP supports inspection of the model features that influenced an assessment [5], while an explicit rule trace records how predictive and temporal evidence was converted into an operational action. This combination supports accountable decision-making without presenting the model output as a confirmed diagnosis.

2.3 Final system positioning

FleetCare AI is positioned as an end-to-end predictive-maintenance decision-support system for commercial fleet operations. Its purpose is to help fleet and maintenance teams allocate limited inspection capacity more effectively while preserving human authority over inspection and repair decisions. It prioritises vehicles and explains the supporting evidence, but it does not claim to identify a physical failure mechanism or autonomously authorise maintenance.

The system follows the reasoning chain:

past vehicle data → C4 ranking and stage probabilities → temporal rules → action and evidence → human decision. (1)

This positioning distinguishes FleetCare AI from a standalone prediction model. Its contribution is the connection between predictive assessment, explicit reasoning, traceable evidence, and a user-facing maintenance workflow. The specific implemented capabilities and boundaries of the completed system are defined in the following section.

3 Project Scope

3.1 Implemented capabilities

The completed project covers the full decision-support path from stored vehicle data to a traceable maintenance action. The implemented scope includes the following capabilities:

1. **Endpoint-aligned C4 prioritisation.**
   Each eligible vehicle is scored at its latest available readout. A specialist binary XGBoost–LightGBM ensemble ranks the active fleet according to near-term C4 risk. The resulting percentile rank is used to allocate limited inspection capacity.
2. **Five-stage and multi-horizon assessment.**
   A separate five-class XGBoost model produces the probability vector $[P(C0), \ldots, P(C4)]$ and the displayed repair-proximity state. The class probabilities are combined into coherent cumulative Risk6, Risk12, Risk24, and Risk48 values for vehicle-level communication.
3. **Temporal rule-based reasoning.**
   The rule engine analyses up to five recent prediction snapshots. It evaluates data sufficiency, persistence, rising risk, fleet-relative deterioration, C4-ranking volatility, and disagreement between the specialist ranking models. Ordered IF–THEN rules convert this evidence into one of the following actions:

   `CONTINUE`, `MONITOR`, `ENHANCED_MONITOR`, `INSPECT`, `PRIORITY_INSPECT`, `MANUAL_REVIEW`, or `COLLECT_MORE_DATA`.

4. **Production inference and database persistence.** The database-backed inference pipeline reads active vehicles that are not currently under maintenance from PostgreSQL, reconstructs the frozen 534-feature schema, scores the latest readout, calculates temporal evidence, applies the rule engine, and writes the completed assessment back to the database. Stored outputs include model probabilities, fleet percentiles, predicted states, temporal indicators, actions, reasons, rule traces, and ranking information.
5. **Model and decision explainability.**
   SHAP identifies the strongest feature contributions from the C4 XGBoost component. These feature-level explanations are stored separately from the rule trace, which records the explicit condition responsible for the operational action. The system therefore distinguishes between an explanation of the model score and an explanation of the final decision.
6. **Fleet and vehicle decision-support interface.**
   A FastAPI backend and React dashboard expose fleet-level priorities, action distributions, maintenance-capacity summaries, high-risk alerts, vehicle details, current probabilities, C4-ranking history, matched rules, action reasons, and SHAP evidence. The interface allows users to move from an overview of the fleet to the evidence associated with an individual vehicle.
7. **Retrieval-augmented assistant.**
   The assistant supports two controlled retrieval paths. Vehicle-specific questions retrieve stored assessments, recent risk history, rule evidence, and SHAP factors from the structured database. Maintenance, policy, and project-knowledge questions retrieve relevant passages from a pgvector knowledge base. Qwen uses Function Calling to select the appropriate tool and generate a response from the retrieved evidence. The assistant explains existing assessments but does not recalculate predictions or modify maintenance actions.

3.2 Scope boundaries and limitations

The following boundaries define how the completed system and its outputs should be interpreted:

- **Anonymised operational inputs.**
  The dataset does not reveal the physical meaning of its operational variables or vehicle specifications. SHAP can therefore identify influential feature IDs and the direction of their effects, but it cannot name a mechanical component or confirm a physical failure cause.
- **Repair proximity rather than fault diagnosis.**
  The model estimates proximity to a recorded maintenance event. A high-risk score or inspection action does not establish that a vehicle has a confirmed fault, nor does it identify the repair that should be performed.
- **Decision support rather than autonomous control.**
  The generated actions prioritise attention and investigation. They do not automatically authorise maintenance, remove a vehicle from service, order replacement parts, or replace qualified maintenance personnel.
- **Probability interpretation.**
  The five-class outputs form a coherent probability distribution within the trained model, but they have not been established as calibrated field failure rates for a deployed commercial fleet. The specialist C4 percentile is a fleet-relative ranking rather than an absolute probability of failure.
- **Policy-threshold validation.**
  Rule thresholds were evaluated using the benchmark data and the project's defined inspection objectives. Before operational adoption, these thresholds would require review by fleet managers, maintenance specialists, and safety personnel using actual workshop capacity, vehicle criticality, and operating policy.
- **Knowledge-base applicability.**
  Retrieved maintenance and policy information must be interpreted according to its source, jurisdiction, review date, and operational applicability. The assistant is required to remain within retrieved evidence, but final interpretation remains the responsibility of the user.
- **Deployment requirements.**
  Use in a live fleet would require prospective validation on operational data, monitoring for input and performance drift, security and access controls, version management for models, rules, and knowledge sources, and integration with existing telematics and workshop-management systems.

4 Data Collection and Preparation

FleetCare AI uses historical vehicle data to learn repair-proximity patterns and a curated document corpus to support grounded maintenance explanations. This section focuses on the collection and preparation of the predictive dataset, including target construction, longitudinal history, leakage control, and feature generation. The preparation and retrieval of the document knowledge base are described separately in Section 5.6.

4.1 Data Source and Characteristics

The project uses the publicly available SCANIA Component X dataset, collected by Scania CV AB and distributed through Researchdata.se under the Creative Commons Attribution 4.0 International licence.[1] The dataset combines longitudinal operational measurements, vehicle specifications, and repair-related information for an anonymised truck engine component. The accompanying data descriptor is provided by Kharazian et al. (2025).[2]

[1] Official dataset record: https://researchdata.se/en/catalogue/dataset/2024-34.

[2] Kharazian et al., "SCANIA Component X dataset: a real-world multivariate time series dataset for predictive maintenance," *Scientific Data*, 12, 493 (2025). https://doi.org/10.1038/s41597-025-04802-6.

Table 2: Dataset inventory recomputed from the supplied operational CSV files. Readouts are repeated observations, whereas vehicles are distinct entities within each partition.

Partition	Vehicles	Operational readouts
Training	23,550	1,122,452
Validation	5,046	196,227
Test	5,045	198,140

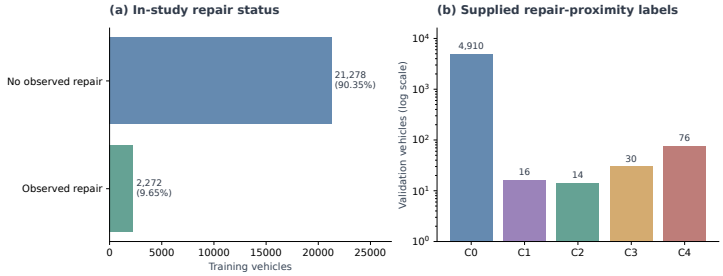


Fig. 1: Two distinct measures of data imbalance: (a) observed repair status among training vehicles and (b) supplied C0–C4 labels among validation vehicles. Panel (b) uses a logarithmic count axis to make minority classes visible. These are source-data counts, not predicted classes; the two panels describe different populations and label definitions.

Data organisation and analysis scope Exploratory analysis was conducted directly on the supplied CSV files, following the data-inspection dimensions identified in the project's exploration script: vehicle coverage, readout frequency, repair status, and missingness. The statistics below describe the source data before sampling, feature engineering, or model-specific preprocessing; they are not model evaluation results. Detailed temporal and missingness analysis uses the training partition, while the supplied validation labels are summarised separately to describe the classification task. Test labels are excluded from this descriptive data analysis and are used only for the final model evaluation reported in Section 5.3.

The operational table contains 105 feature columns, excluding vehicle identifiers and time. Their prefixes identify 14 anonymised variable groups; multiple columns can therefore belong to the same underlying variable rather than represent independent sensors. Eight additional vehicle-specification fields provide static context. Training vehicle identifiers match across the operational, specification, and repair tables. No duplicate vehicle–time pairs were found in the training readouts, and neither the validation nor test population overlaps with the training vehicle identifiers. These checks support reliable table joins and preservation of the supplied partition boundaries.

Repair-event imbalance and censoring Of the 23,550 training vehicles, 2,272 (9.65%) have an observed in-study repair and 21,278 (90.35%) do not. Figure 1(a) shows this vehicle-level imbalance. Absence of an observed repair does not establish that a vehicle remains repair-free after observation ends. Such records are right-censored: their subsequent repair time is unknown. Consequently, a readout should only receive a no-repair-within-horizon label when sufficient follow-up is available; otherwise, its status for that horizon remains uncertain.

The validation labels show a more pronounced imbalance across repair-proximity states: C0 contains 4,910 vehicles (97.30%), while C1–C4 contain 16, 14, 30, and 76 vehicles, respectively. These labels describe proximity at the validation assessment point, unlike the training repair indicator, which records whether a repair occurred during the study. They should therefore not be interpreted as evidence of a change in repair incidence between partitions. The small minority-class counts motivate class-sensitive evaluation and cautious interpretation of subsequent estimates, rather than reliance on overall accuracy alone.

Longitudinal coverage and irregular sampling Training vehicles contribute between 5 and 303 readouts, with a median of 43 and an interquartile range of 28–64 (Fig. 2). Treating every readout as an equally independent sample would give greater influence to vehicles with longer or denser histories. This observation motivates vehicle-aware preparation and explicit separation of vehicle-level and readout-level statistics.

Consecutive-readout intervals have a median of 4.4 operational time units and an interquartile range of 1.6–6.2; the maximum is 387.8. A fixed number of previous readouts therefore does not correspond to

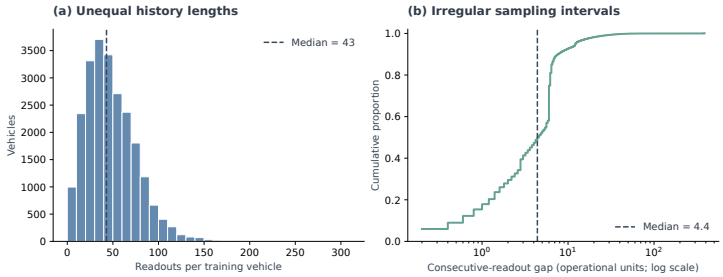


Fig. 2: Temporal coverage in the training data: (a) distribution of readout counts across 23,550 vehicles and (b) cumulative distribution of 1,098,902 within-vehicle consecutive-readout intervals after sorting by time. Panel (b) uses a logarithmic horizontal axis and includes the full interval range. Dashed lines indicate medians.

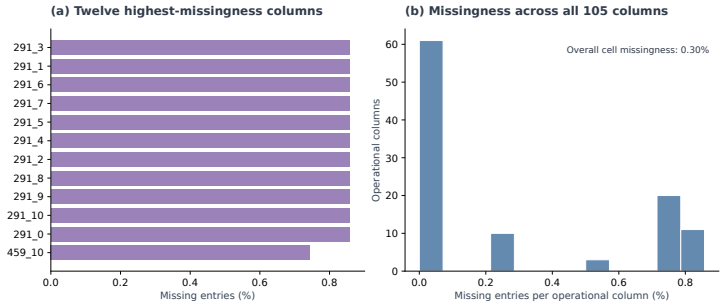


Fig. 3: Missingness before preprocessing in the training operational table: (a) twelve columns with the highest missing-entry proportions and (b) the distribution of these proportions across all 105 feature columns. Overall cell missingness is approximately 0.301%. Column names retain their original anonymised identifiers.

a fixed operational duration. In the final representation, lags of 1, 3, and 5 refer to earlier observations rather than fixed time windows. Exposure-normalised rates reduce the influence of accumulated vehicle usage, while irregular spacing remains an important limitation when interpreting recent changes. The benchmark time scale is retained without converting it into calendar days or assigning an unsupported physical meaning.

Missingness and anonymised feature representation Across all training operational feature cells, 0.301% are missing. At least one missing entry occurs in 104 of the 105 columns, although the largest column-level missingness is only approximately 0.858%. No missing entries were found in the eight training specification fields. Figure 3 shows both the highest-missingness columns and the distribution across all operational columns. The repeated rates among columns with the same prefix warrant considering grouped feature availability rather than assuming each column has an independent missingness pattern.

Anonymisation introduces a separate limitation: feature identifiers cannot reliably be translated into named physical measurements or failure mechanisms. Subsequent explanations will therefore describe model associations with anonymised features rather than claim a causal diagnosis.

These observations motivate censoring-aware targets, vehicle-level preparation, past-only temporal features, native tree-based handling of missing and undefined features, and class-sensitive evaluation.

4.2 Target construction and censoring

For a vehicle i with an observed maintenance event at operational time T_i, time to event at observation t is

$$\mathrm{TTE}_{i,t} = T_i - t. \tag{2}$$

The five class labels are constructed as

$$y_{i,t} = \begin{cases} 0, & \mathrm{TTE}_{i,t} > 48, \\ 1, & 24 < \mathrm{TTE}_{i,t} \leq 48, \\ 2, & 12 < \mathrm{TTE}_{i,t} \leq 24, \\ 3, & 6 < \mathrm{TTE}_{i,t} \leq 12, \\ 4, & 0 \leq \mathrm{TTE}_{i,t} \leq 6. \end{cases} \tag{3}$$

Observations after the recorded event are excluded. For a censored vehicle with follow-up endpoint L_i, an observation is treated as a safe C0 example only when $L_i - t > 48$. Censored observations with insufficient remaining follow-up are excluded rather than incorrectly labelled as C0. Event records and derived TTE values are used only to construct labels and evaluation outcomes; they are never model inputs.

4.3 Leakage control and endpoint alignment

Readouts are ordered by `vehicle_id` and operational `time_step`. Every feature for an observation is computed from that readout and earlier readouts of the same vehicle only. Splits are vehicle-based, preventing closely related observations from one vehicle from appearing in both training and evaluation partitions. Vehicle identifiers are retained for grouping and traceability but are excluded from the feature matrix.

The final C4 specialist uses one endpoint example per vehicle. This aligns training with production, where the system must rank every vehicle once at its latest available readout. The auxiliary five-class stage model retains the broader vehicle-aware training representation needed to estimate the complete stage distribution, but it uses the same frozen feature schema and vehicle-level separation.

4.4 Final feature representation

The operational variables consist of cumulative stand-alone counters and histogram bins. Direct use of their raw levels would primarily reflect accumulated operating exposure, making vehicles with different operating histories difficult to compare. The final pipeline therefore converts the source variables into 534 past-only model features:

- 14 exposure-normalised rate features;
- 97 within-histogram proportions;
- 291 recent histogram-mix features calculated over lags of 1, 3, and 5 readouts;
- 42 recent-to-lifetime rate ratios over the same three lags; and
- 90 one-hot encoded vehicle-specification indicators.

The rate features reduce the influence of accumulated operating exposure, while the histogram proportions represent how activity is distributed within each anonymised variable group. Recent histogram-mix features capture short-term changes in operating patterns, and recent-to-lifetime ratios compare current behaviour with the vehicle's longer-term baseline.

Only information available at or before the prediction readout is used. Raw `time_step` and `vehicle_id` are excluded as direct predictors. Missing source readings and undefined derived features are not statistically imputed. They are retained as `NaN` and handled natively by XGBoost and LightGBM during tree construction. A derived value is marked as missing when its denominator is non-positive, the required historical information is unavailable, or a cumulative counter decreases. Missing vehicle-specification records are treated as data-integrity errors. The eight categorical specification fields are one-hot encoded and aligned to the schema fixed during training.

The same transformation code is used during model training and database inference. Feature names and column order are stored in the model metadata, and inference terminates if the reconstructed feature schema differs from the training schema. This contract prevents silent training–serving skew and ensures that the model, database, dashboard, and explanation layers refer to the same feature representation.

5 System Design

5.1 Overall Architecture

FleetCare AI is designed as a hybrid intelligent reasoning system that combines data-driven prediction, historical risk evidence, knowledge-based reasoning, and explainability within a human-in-the-loop decision-support framework.

Figure 4 presents the implemented architecture. Historical operational readouts and vehicle specifications first pass through preprocessing and temporal feature construction. The predictive layer then estimates repair-proximity states and model-derived risk scores for available vehicle snapshots.

Historical model outputs provide additional evidence about how a vehicle's predicted risk has evolved over time. This includes fleet-relative risk, persistence of elevated risk, recent predicted-state trajectories,

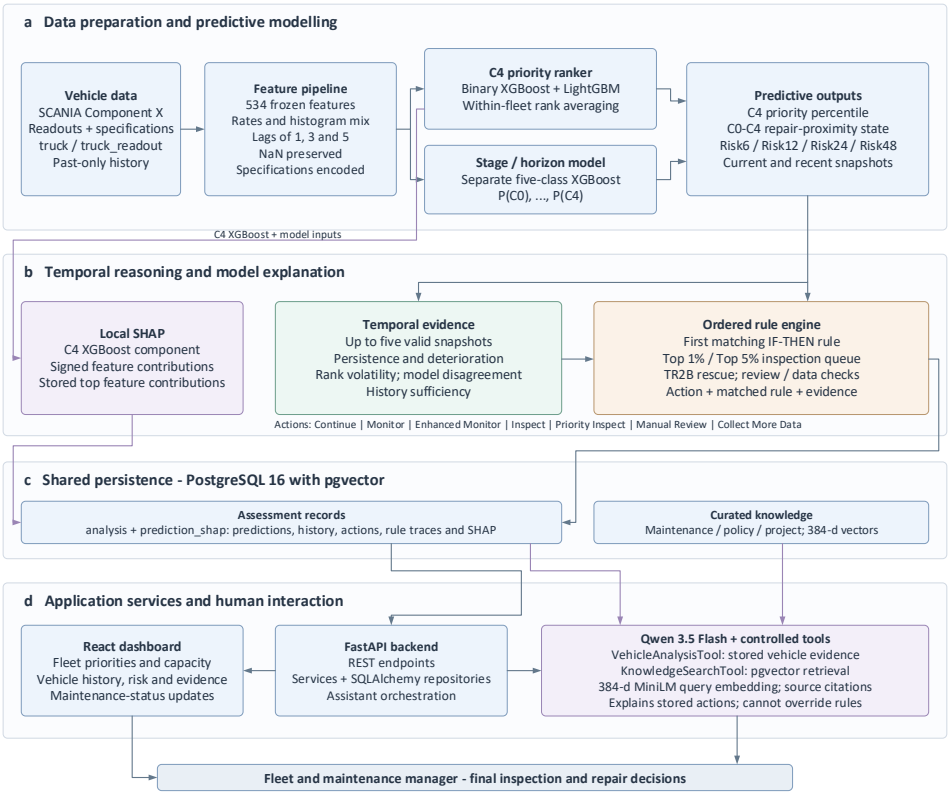


Fig. 4: FleetCare AI system architecture. Vehicle histories are transformed into repair-proximity predictions and multi-horizon risk scores. Current predictions and temporal evidence feed an explicit rule engine, while SHAP provides a separate model-explanation path. Shared backend records support the implemented dashboard and retrieval-augmented assistant; final maintenance decisions remain with human users.

relative deterioration, and the amount of historical information available. The knowledge-based reasoning layer combines the current prediction with this supporting evidence to generate an appropriate maintenance recommendation.

Explainability is incorporated at both the predictive and decision levels. SHAP is used to examine which features contribute to the machine-learning prediction, while the rule engine produces an explicit reasoning trace explaining why a particular maintenance recommendation is generated.

The resulting predictions, risk indicators, explanations, and recommendations are structured for delivery through backend services to a fleet-management dashboard and a RAG-based assistant. These user-facing components are intended to help fleet and maintenance managers prioritise vehicles and inspect the evidence behind each recommendation while retaining human control over final maintenance decisions.

5.2 Database Design

FleetCare AI uses PostgreSQL as the shared persistence layer connecting vehicle data, predictive inference, rule-based decisions, explainability, and knowledge retrieval. This allows the backend, dashboard, and AI assistant to access the same stored assessment and supporting evidence.

The principal operational tables are:

- `truck`: stores the vehicle identifier, eight specification fields, maintenance status, and active-state information;
- `truck_readout`: stores the time-indexed operational observations used to construct the predictive-model features;
- `analysis`: stores each vehicle assessment, including the prediction time, predicted maintenance stage, horizon-risk scores, fleet-relative percentiles, temporal indicators, matched rule, recommended action, and supporting reason;

– `prediction_shap`: stores the highest-ranked SHAP feature contributions associated with each assessment.

The knowledge-retrieval subsystem uses two additional tables:

– `knowledge_document`: stores document-level information, including the title, source type, publisher, source URL, and jurisdiction;
– `knowledge_chunk`: stores retrievable passages, citation identifiers, source metadata, and 384-dimensional pgvector embeddings.

The principal database relationships are:

$$\begin{aligned} \texttt{truck} &\longrightarrow \texttt{truck_readout} && \text{(one-to-many)},\\ \texttt{truck} &\longrightarrow \texttt{analysis} && \text{(one-to-many)},\\ \texttt{analysis} &\longrightarrow \texttt{prediction_shap} && \text{(one-to-many)},\\ \texttt{knowledge_document} &\longrightarrow \texttt{knowledge_chunk} && \text{(one-to-many)}. \end{aligned} \tag{4}$$

A vehicle can therefore retain multiple historical readouts and multiple model assessments, while each assessment can retain several SHAP contribution records. Assessments are appended rather than overwritten, preserving the vehicle's prediction and decision history. Similarly, each knowledge document can be divided into multiple retrievable chunks.

This structure provides the two evidence paths used by the AI assistant. `VehicleAnalysisTool` retrieves structured vehicle evidence from `truck`, `analysis`, and `prediction_shap`, including the predicted state, risk history, rule-engine action, supporting reason, and SHAP factors. `KnowledgeSearchTool` performs pgvector similarity search over `knowledge_chunk` and uses `knowledge_document` to return the corresponding source information and citation metadata. Through Function Calling, the assistant can use either tool independently or combine both when a question requires vehicle-specific evidence together with documented maintenance or policy guidance.

5.3 Predictive Model

Prediction objective and operational framing FleetCare AI is designed as an inspection-prioritisation system rather than an automatic diagnostic system. The five repair-proximity stages and their target construction are defined in Table 1 and Equation 3. Among these stages, C4 is the rarest and most operationally urgent because it represents the final 0–6 units before the recorded maintenance event.

A fleet operator can inspect only a limited number of vehicles. The primary modelling objective is therefore to place as many true C4 vehicles as possible near the top of a fixed-capacity inspection queue. Overall five-class accuracy is not used as the principal selection criterion because it is dominated by the majority C0 class and may remain high even when urgent C4 vehicles are missed.

The vehicle-level separation, censoring controls, past-only feature construction, and endpoint-aligned training protocol are described in Section 4. The final specialist ranker scores one endpoint observation per vehicle, matching the production setting in which each eligible vehicle is assessed at its latest available readout.

Model input The predictive models use the frozen 534-feature representation described in Section 4.4. This representation combines exposure-normalised operational behaviour, recent temporal changes, comparison with lifetime behaviour, and one-hot encoded vehicle specifications. Feature construction is identical during training and production inference.

Final model architecture The final package contains three fitted components. A binary XGBoost model and a binary LightGBM model independently estimate C4 risk. Their raw probabilities are not averaged directly because the two libraries can produce differently calibrated probability scales. Instead, each probability vector is converted to a within-fleet percentile rank. The two ranks are averaged and ranked again:

$$s_i = \mathrm{rank}_{\%}\left(\frac{\mathrm{rank}_{\%}(p_i^{\mathrm{XGB}}) + \mathrm{rank}_{\%}(p_i^{\mathrm{LGB}})}{2}\right), \tag{5}$$

where $s_i \in (0, 1]$ is the final C4 priority score for vehicle i. Rank averaging makes the queue robust to probability-scale differences while preserving the relative ordering produced by each learner. The binary models use the complete 534-feature representation and the selected flexible tree profile. Moderate C4 weighting with exponent 0.75 is used during fitting.

A separate five-class XGBoost model supplies the predicted maintenance stage and a coherent class-probability vector $[P(C0), \ldots, P(C4)]$. The displayed horizon risks are cumulative sums of this vector:

$$\text{Risk6} = P(C4), \tag{6}$$
$$\text{Risk12} = P(C3) + P(C4), \tag{7}$$
$$\text{Risk24} = P(C2) + P(C3) + P(C4), \tag{8}$$
$$\text{Risk48} = P(C1) + P(C2) + P(C3) + P(C4). \tag{9}$$

This distinction is important in the implementation. The database field `risk6_score` stores the five-class model's estimated probability $P(C4)$, whereas `risk6_percentile` stores the binary XGBoost–LightGBM ensemble rank used for prioritisation. Consequently, the underlying five-class probabilities remain non-negative and sum to one, while the cumulative horizon risks satisfy Risk6 $\leq$ Risk12 $\leq$ Risk24 $\leq$ Risk48.

Model evaluation The official Test split contains 5,045 vehicles, of which only 60 are labelled C4. Table 3 compares the original five-class baseline with the final specialist ensemble. At a 5% inspection budget, the final model places 20 C4 vehicles among the first 253 vehicles, compared with four for the baseline. Precision at 5% therefore rises from 1.58% to 7.91%, while recall rises from 6.67% to 33.33%. Average Precision and ROC–AUC also improve, but the fixed-budget result is the most directly relevant operational measure.

Table 3: C4 ranking performance on the official Test split.

Metric	Original baseline	Final model
Average Precision	0.0205	**0.0552**
ROC–AUC	0.6722	**0.7907**
C4 found in Top 5%	4/60	**20/60**
Precision at Top 5%	1.58%	**7.91%**
Recall at Top 5%	6.67%	**33.33%**

The model output should still be interpreted as a prioritised inspection queue, not a confirmed fault diagnosis. Even after improvement, most vehicles in the Top-5% queue are not C4. This is expected under the extreme class prevalence and is the reason that model scores are combined with an explicit decision layer rather than presented as automatic maintenance commands.

5.4 Rule engine

The rule engine translates model evidence into operational actions. Rules are evaluated in the order shown in Table 4, and the first matching rule determines the action. TR1 and TR2 preserve the model-led immediate-inspection queue: vehicles in the first 1% receive `PRIORITY_INSPECT`, while the remainder of the first 5% receive `INSPECT`. Subsequent rules use a rolling history of up to five recent prediction snapshots to evaluate data availability, ensemble-rank volatility, model disagreement, persistence, and deterioration. Percentile thresholds are calculated within the currently eligible fleet, defined as active vehicles that are not already in maintenance.

TR2B is the only rule permitted to extend the immediate-inspection queue beyond the exact model Top 5%. On the Test split, the model-only Top-5% queue contains 253 vehicles and identifies 20 of the 60 C4 cases. TR2B adds 22 vehicles and identifies two additional C4 cases, producing a 275-vehicle queue containing 22 C4 cases, with 8.00% precision and 36.67% recall.

Selecting the next 22 vehicles directly from the model ranking also produces 22 C4 cases in total. TR2B therefore does not demonstrate an equal-capacity improvement over direct model ranking. Its value is

Table 4: Ordered production rules.

Rule	First matching condition	Action
TR1	Vehicle is in the first 1% of the final C4 ensemble ranking	`PRIORITY_INSPECT`
TR2	Vehicle is in the remainder of the first 5% of the final C4 ensemble ranking	`INSPECT`
TRD	No valid prediction is available	`COLLECT_MORE_DATA`
TR0	Outside the protected model Top 5%, only one valid prediction is available	`COLLECT_MORE_DATA`
TRV	C4 ensemble-rank volatility is at or above P95 and XGBoost–LightGBM rank disagreement is at or above P90	`MANUAL_REVIEW`
TR2B	Outside the model Top 5%, C4 rank $\geq$ P86, Risk12 rank $\geq$ P96, and Risk12 deterioration rank $\geq$ P62	`INSPECT`
TR4	C4 rank $\geq$ P90 and the recent C4 rank is persistent or rising	`ENHANCED_MONITOR`
TR3	C4 rank $\geq$ P80 and the latest positive C4-rank change is at or above P95 relative to the eligible fleet	`ENHANCED_MONITOR`
TR5	C4 rank $\geq$ P80 and the recent C4 rank is persistent or rising	`MONITOR`
TR6	No escalation condition is met	`CONTINUE`

instead procedural and explanatory: it defines a transparent rescue condition requiring simultaneous evidence from the C4 ranking, the 12-unit risk ranking, and recent Risk12 deterioration.

The remaining rules do not attempt to outperform the model within the immediate inspection budget. They organise lower-priority monitoring, route volatile and model-disagreement cases to manual review, and request additional observations for insufficient-history vehicles outside the protected model Top-5% queue.

5.5 Inference, persistence, and explainability

The production inference script reads operational history and vehicle specifications directly from PostgreSQL, filters to active vehicles outside maintenance, and scores each vehicle at its latest readout. It reconstructs the same 534-feature schema stored in the model metadata and refuses inference if the feature order differs. For each output assessment, the pipeline constructs a rolling history of up to five valid prediction snapshots to support the temporal rules. The script then stores the predicted stage, horizon probabilities, fleet percentiles, action, matched rule, action reason, evidence string, and rule trace in the `analysis` table.

For local model explainability, SHAP contributions are computed from the C4 XGBoost component. The five features with the largest absolute contributions are stored for each prediction in `prediction_shap`. A positive SHAP value indicates that the feature pushes the XGBoost component toward C4; a negative value pushes it away from C4. Since the operational variables are anonymised, these explanations identify influential feature IDs and temporal patterns but must not be presented as named mechanical causes. The explanation layer therefore supports model auditing and inspection prioritisation rather than component-level diagnosis.

5.6 RAG-Based Assistant

The FleetCare AI assistant provides natural-language access to vehicle assessments and maintenance references. It helps users understand stored predictions and recommendations by retrieving supporting evidence before generating an explanation. The implementation combines Qwen 3.5 Flash, predefined backend tools, and PostgreSQL-based storage.

RAG Implementation The assistant uses two retrieval paths, depending on whether a question concerns a specific vehicle or documented maintenance knowledge.

Vehicle-specific evidence For questions about a vehicle's condition, recent risk history, or recommended action, `VehicleAnalysisTool` retrieves the latest assessment and up to five stored analysis records from PostgreSQL. The returned evidence includes the predicted state, Risk6 score and percentile, selected temporal indicators, maintenance action, matched rule, reason, and supporting evidence. Up to five SHAP factors provide additional information about feature contributions to the prediction.

This path allows the assistant to explain an existing assessment using recorded vehicle data. The tool returns selected assessment fields rather than the complete database record; explanations must therefore remain within the information actually retrieved.

Document-based knowledge For questions about maintenance guidance, inspection policies, or project concepts, `KnowledgeSearchTool` retrieves passages from the document knowledge base. The distributed corpus contains five sources and 274 chunks: 98 maintenance chunks, 168 policy chunks, and eight project-definition chunks.

The integrated backend encodes the question using a multilingual 384-dimensional embedding model and searches the stored vectors through pgvector cosine similarity. Retrieval requires a knowledge category and returns at most five passages with similarity of at least 0.30. Review-date filters exclude overdue material, while policy queries admit Singapore and global references. Each retrieved passage retains its citation identifier, source URL, jurisdiction, and applicability metadata.

Tool orchestration and response generation Qwen selects the appropriate retrieval tool through Function Calling. The backend orchestrator executes the requested tool, returns its results to the model, and allows the model to generate an explanation from the available evidence. Questions combining vehicle analysis and general guidance can use both retrieval paths.

The orchestrator permits up to five tool-calling rounds within a request. Its prompt instructs the model to use retrieved vehicle facts, cite supporting documents, and acknowledge insufficient evidence. Citation compliance is prompt-guided; the implementation does not include a separate response-level citation validator.

Operational Scope and Evaluation The assistant serves as the explanation and retrieval interface of the integrated FleetCare AI system. For vehicle-specific questions, it retrieves stored predictions, maintenance actions, rule traces, temporal evidence, and SHAP factors from PostgreSQL. It does not recalculate predictions, execute or modify the rule engine, or replace the maintenance actions produced by the inference pipeline.

For document-based questions, the assistant retrieves relevant passages from the maintenance and policy knowledge base before generating a response. The current endpoint processes each request independently and does not retain conversational memory across requests.

The assistant was evaluated using ten document questions and three synthetic vehicle-assessment cases. The evaluation covered retrieval quality, answer grounding, citation behaviour, vehicle-tool execution, and consistency with stored assessment records. Section 7 presents the evaluation configuration, results, integration checks, and remaining limitations.

5.7 Application Layer

The application layer translates the outputs of the predictive and rule-based reasoning components into an interactive decision-support system for fleet managers and maintenance staff. It provides three main user-facing capabilities: fleet-level monitoring and maintenance planning, vehicle-level risk analysis and explainability, and natural-language assistance.

The implemented system integrates a React frontend, a FastAPI backend, PostgreSQL for structured data storage, and a Qwen-powered assistant for retrieving and explaining vehicle assessments and maintenance knowledge.

Application architecture The application adopts a modular frontend–backend architecture, separating presentation, API handling, business logic, and data access.

The frontend is developed using React and TypeScript, with Vite supporting development and application bundling. It communicates with the backend through RESTful HTTP requests and renders the returned data as interactive tables, charts, and conversational responses.

The backend is implemented using FastAPI and follows an API–Service–Repository architecture. The API layer handles HTTP requests, validates input parameters, and returns structured responses. The service layer coordinates application logic, including fleet-level aggregation, maintenance statistics, and vehicle information retrieval. The repository layer uses SQLAlchemy to access PostgreSQL.

The AI assistant follows a separate orchestration path, where Qwen selects predefined retrieval tools through Function Calling. Tool execution and database access remain controlled by the backend.

This separation allows the user interface, business services, database operations, and assistant tools to be maintained independently. Table 5 summarises the main responsibilities of each component.

Table 5: Main components of the FleetCare AI application layer.

Component	Responsibility
React frontend	Presents fleet statistics, vehicle assessments, maintenance priorities, and conversational interactions.
FastAPI	Exposes RESTful endpoints and manages request validation and response delivery.
Business services	Coordinates fleet aggregation, maintenance planning, vehicle retrieval, and application logic.
SQLAlchemy repositories	Accesses stored vehicle information, prediction records, and supporting explanations in PostgreSQL.
AI assistant	Coordinates Qwen Function Calling and backend retrieval tools for vehicle and knowledge queries.

Fleet dashboard and maintenance planning The Fleet Dashboard provides a consolidated view of fleet condition and maintenance priorities. It is designed to help fleet managers and maintenance staff identify vehicles requiring attention and assess available maintenance resources.

As shown in Fig. 5, the dashboard presents fleet overview statistics, including active vehicles, vehicles at risk, maintenance capacity, and the average fleet risk score. The Risk Snapshot visualises the distribution of recommended maintenance actions, while Top Risk Alerts highlight vehicles with the highest fleet-relative risk rankings.

The Maintenance Plan Summary presents four fleet-level planning indicators derived from the rule-engine actions. *Immediate* corresponds to `PRIORITY_INSPECT`; *Short Term* combines `INSPECT` and `ENHANCED_MONITOR`; *Monitor* combines `MONITOR` and `CONTINUE`; and *Manual Review* corresponds to `MANUAL_REVIEW`. The `COLLECT_MORE_DATA` action remains a separate vehicle-level outcome and is not included in the current Maintenance Plan Summary. These planning indicators simplify the fleet-level presentation without modifying the underlying rule-engine decisions.

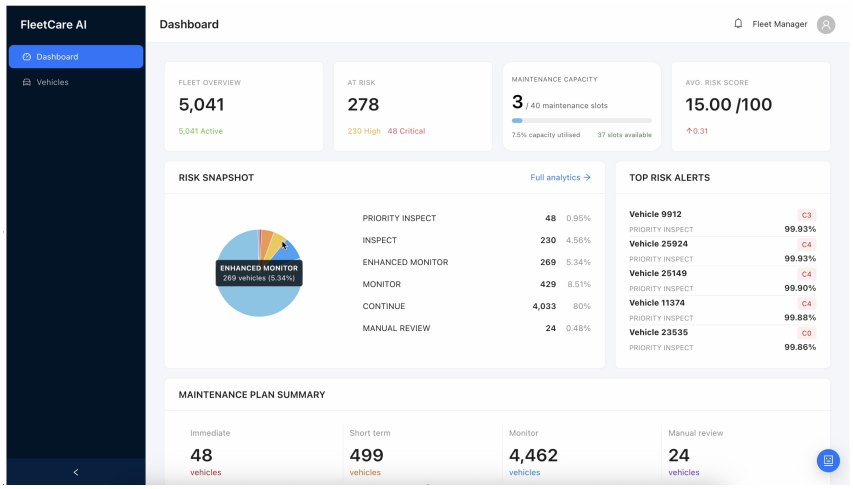


Fig. 5: FleetCare AI dashboard showing fleet-level risk indicators, maintenance capacity, priority alerts, and maintenance planning summaries.

The dashboard execution sequence is illustrated in Fig. 6. When the dashboard is opened, the React frontend requests fleet statistics from the FastAPI backend. The Dashboard Service coordinates data retrieval and aggregation through the Dashboard Repository, which queries PostgreSQL using SQLAlchemy.

For vehicle-based risk and action summaries, the backend retrieves the latest available assessment for each eligible vehicle. Eligibility is determined by vehicle activity status and whether the vehicle is already undergoing maintenance. The retrieved records are aggregated into fleet statistics, risk distributions, and priority alerts.

The resulting information is returned as structured JSON responses and rendered by the frontend. Unlike the AI assistant, the dashboard does not require language-model inference, since its statistics are derived directly from stored vehicle assessments.

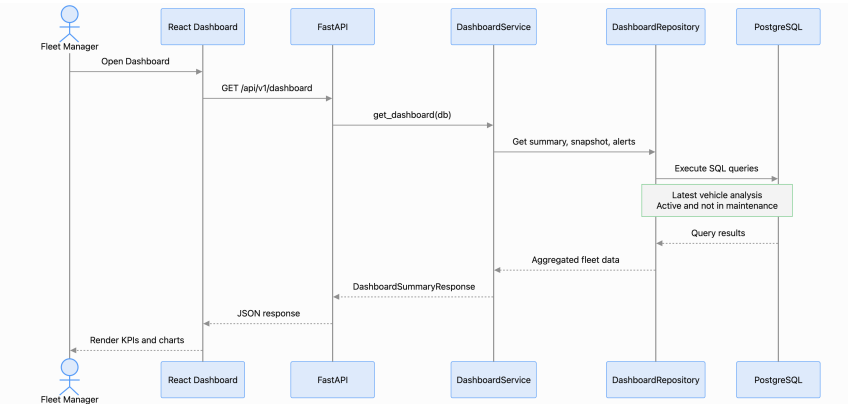


Fig. 6: Dashboard execution sequence illustrating data retrieval and aggregation through React, FastAPI, business services, repositories, and PostgreSQL.

Vehicle-level analysis and explainability The vehicle module allows fleet managers and maintenance staff to examine individual assessments alongside the underlying prediction and reasoning evidence. Rather than presenting maintenance priorities as isolated labels, the interface links each recommended action to the vehicle's predicted repair proximity, recent risk history, and available explanations.

Figure 7 shows the Vehicle Risk Overview page, where vehicles are listed with their predicted classes, fleet-relative risk scores, recommended actions, and maintenance status. Staff can search by vehicle identifier or filter the list by predicted class to narrow down the vehicles requiring further review. The maintenance status can also be updated directly from this page. Vehicles marked as undergoing maintenance are excluded from subsequent fleet-level prioritisation, preventing them from occupying additional inspection capacity while work is already in progress.

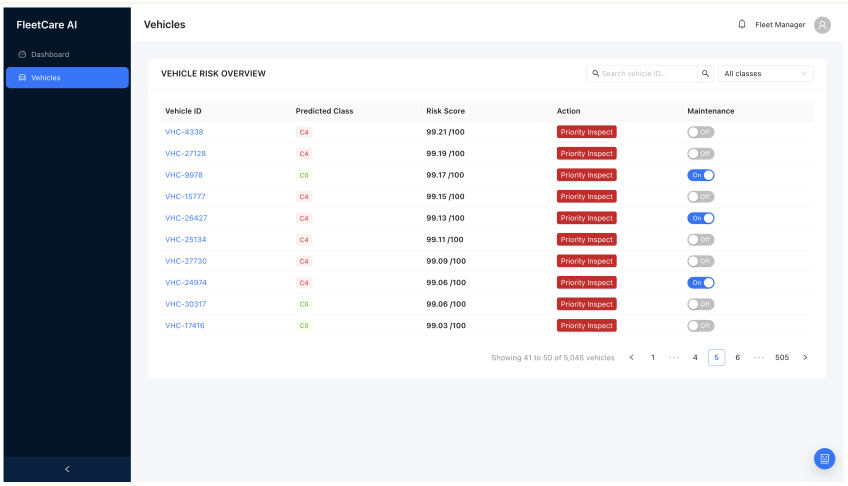


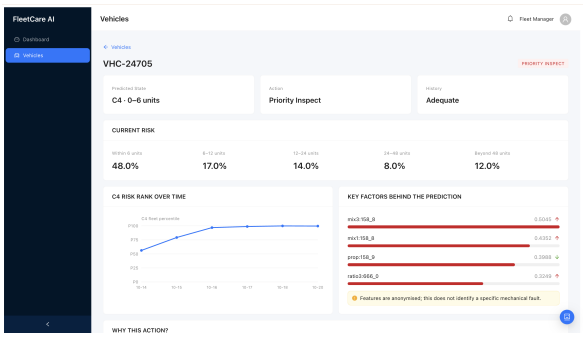
Fig. 7: Vehicle Risk Overview showing predicted classes, fleet-relative risk rankings, maintenance actions, and maintenance status controls.

Selecting a vehicle provides a more detailed account of its current assessment. As illustrated in Fig. 8, the detail page brings together the predicted repair-proximity state, multi-horizon risk indicators, recent C4 risk rankings, and the reasoning behind the recommended action. The historical chart uses fleet-

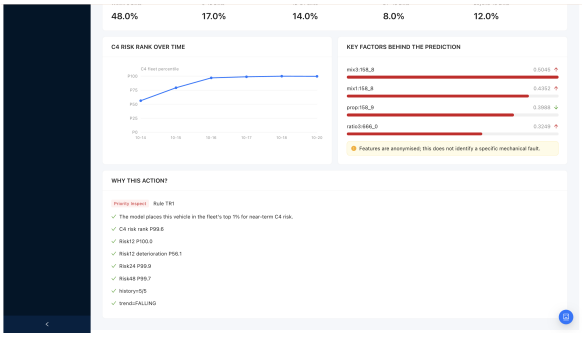
relative percentiles rather than raw risk scores, allowing staff to see how the vehicle's position within the assessed fleet has changed across successive predictions.

Vehicle `VHC-24705` provides an example of how model predictions are translated into maintenance recommendations. The predictive model classifies the vehicle as C4 and estimates its near-term repair risk through Risk6. The rule engine then evaluates the vehicle's fleet-relative C4 risk ranking and assigns `PRIORITY_INSPECT` under rule TR1, as the vehicle falls within the highest 1% of the fleet. The interface presents the predicted state, risk indicators, and supporting rule evidence, allowing staff to understand how the maintenance recommendation was determined.

The SHAP panel addresses a different question. It shows which anonymised input features contributed most strongly to the model output, whereas the rule evidence explains why the stored assessment was assigned a particular maintenance action. Keeping these explanations separate is important because a feature's predictive contribution does not establish a mechanical fault, and the final action may also depend on fleet-relative and temporal evidence beyond the predicted class.



(a) Current assessment, risk history, and SHAP feature contributions.



(b) Rule TR1 and the evidence supporting the maintenance recommendation.

Fig. 8: Vehicle-level assessment of `VHC-24705`. The interface combines prediction results, historical C4 risk rankings, SHAP explanations, and rule-based evidence for the recommended maintenance action.

The implementation separates vehicle presentation from database access. Figure 9 shows the request sequence for the three operations supported by the module: loading the vehicle list, updating maintenance status, and retrieving an individual vehicle assessment.

When the vehicle page is opened, React requests the available maintenance action categories and the vehicle records through FastAPI. The Vehicle Service retrieves the latest stored assessments from PostgreSQL and returns the results for display and filtering. A maintenance status change follows a separate request, which updates the corresponding flag in the `truck` table.

For an individual vehicle, the service retrieves the prediction history and the SHAP contributions associated with the latest assessment. The returned data are then used to render the risk history, feature explanations, and rule evidence shown on the detail page. This arrangement keeps database queries and record updates within the backend while allowing the frontend to present the same assessment data through different views.

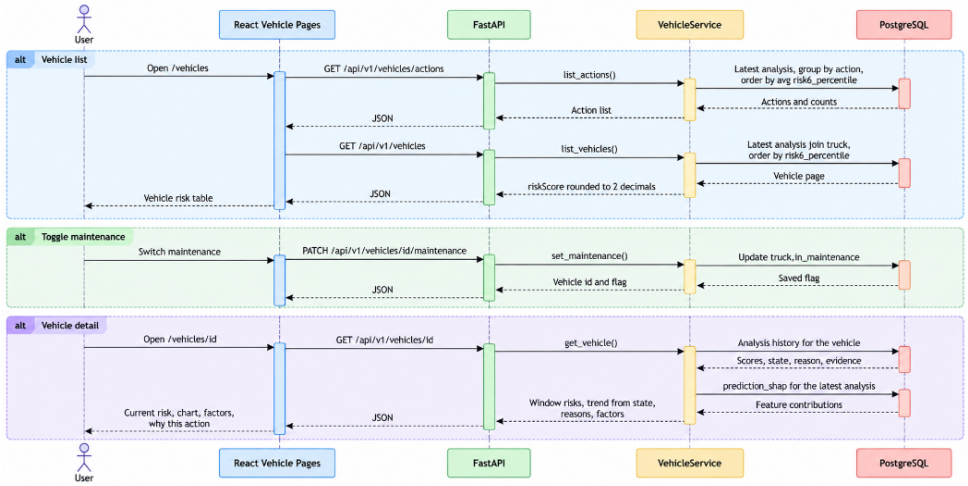


Fig. 9: Vehicle module execution sequence covering vehicle listing, maintenance status updates, and retrieval of prediction history and SHAP explanations.

AI assistant integration The AI assistant provides a conversational interface for accessing vehicle assessments and maintenance references. Unlike general knowledge questions, vehicle-specific queries require information from the latest stored predictions and reasoning results. Allowing the language model to answer such questions from its own knowledge could introduce inconsistencies with the actual vehicle records. The assistant therefore uses backend retrieval tools to obtain the relevant evidence before generating a response.

The implementation uses Qwen 3.5 Flash through an OpenAI-compatible API, with Function Calling connecting the model to two retrieval tools. The `VehicleAnalysisTool` queries PostgreSQL for vehicle predictions, historical risk indicators, maintenance decisions, and SHAP explanations. The `KnowledgeSearchTool` handles questions about maintenance practices and transport policies by encoding the query with a multilingual embedding model and searching the knowledge base through pgvector. These two retrieval paths allow the assistant to distinguish vehicle-specific facts from general supporting knowledge.

Figure 10 shows an example involving vehicle `VHC-24705`. The user asks how the vehicle's risk has changed recently. Rather than reporting only the latest risk score, the assistant summarises the recent prediction history, identifies changes in the vehicle's fleet-relative risk ranking, and explains the current maintenance recommendation. This provides a conversational alternative to manually examining individual assessment records.

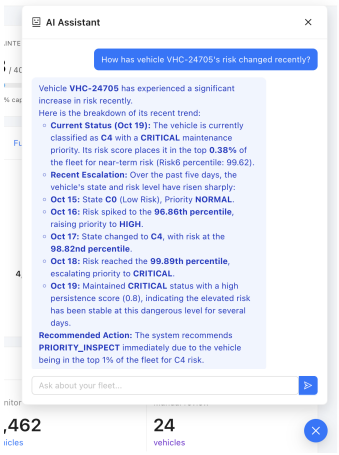


Fig. 10: Vehicle-specific risk explanation generated by the FleetCare AI assistant for vehicle `VHC-24705`.

The interaction sequence is illustrated in Fig. 11. After receiving a question from the React frontend, FastAPI passes it to the Assistant Orchestrator, which sends the message and available tool definitions to

Qwen. The model returns a function call specifying the selected tool and its arguments. The orchestrator then executes the corresponding backend tool and supplies the retrieved results to Qwen for response generation. For knowledge-based queries, the retrieved passages also provide source references that can be included in the answer.

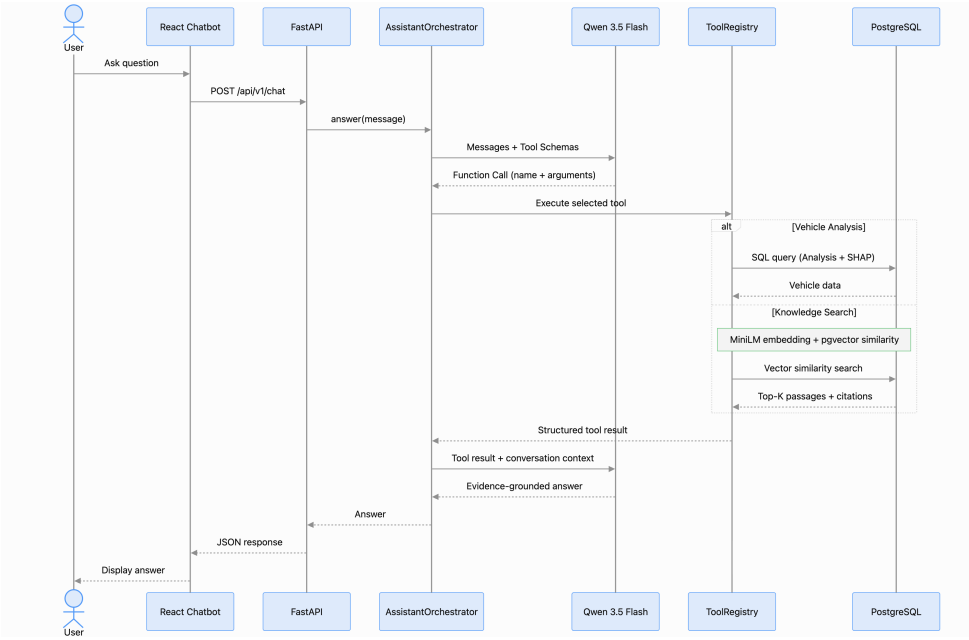


Fig. 11: Execution sequence of the AI assistant, showing tool selection through Function Calling and evidence retrieval from PostgreSQL.

A key design decision is to keep tool execution within the backend rather than allowing the language model to access the database directly. This preserves the separation between the predictive and rule-based decision pipeline and the conversational interface. The assistant explains existing assessments but does not recalculate model predictions or replace maintenance actions assigned by the rule engine. Its retrieval methods and evaluation are discussed separately in the RAG implementation and evaluation sections.

Application integration and extensibility The completed application connects predictive modelling, temporal reasoning, rule-based maintenance decisions, and explainability through shared database records and structured backend interfaces.

The Dashboard, Vehicle Analysis module, and AI Assistant access consistent stored assessment information while serving different user interaction requirements. The dashboard supports fleet-level monitoring, the vehicle module provides detailed assessment inspection, and the assistant supports natural-language access to vehicle evidence and maintenance knowledge.

The modular frontend–backend design also supports future extension. Additional API endpoints, maintenance workflows, and assistant tools can be introduced without fundamentally changing the existing application architecture.

Although the completed system demonstrates an integrated decision-support workflow, deployment in a live commercial fleet would require further validation, authentication, role-based access control, operational monitoring, and integration with live fleet-management systems.

6 Predictive Modelling Experiments

Model development proceeded through a sequence of controlled experiments rather than a single algorithm-selection step. The main experimental variables were the prediction objective, temporal feature representation, sampling strategy, class weighting, tree complexity, probability calibration, ensemble structure, and alignment between training and production inference. The experiments were evaluated primarily at vehicle level because the operational decision concerns which vehicles should enter a limited inspection queue, rather than which individual readouts should receive the correct label.

6.1 Experimental protocol and evaluation criteria

All primary development splits were vehicle based. Readouts from the same vehicle were not divided between model-development and evaluation sets, thereby preventing the model from being evaluated on vehicles whose earlier history had already appeared during training. Early experiments sampled multiple historical observations from each vehicle, whereas later experiments used one endpoint observation per vehicle to reproduce the production setting, in which each active vehicle is scored at its latest available readout.

Several evaluation criteria were considered during development. Five-class accuracy and Macro F1 were initially used to assess general classification performance. However, accuracy was dominated by C0, while Macro F1 remained close to 0.20 because the minority stages contained comparatively few observations. Multi-horizon Average Precision and ROC–AUC were then used to evaluate cumulative risks within 6, 12, 24, and 48 units. Although these metrics were more informative, they still did not directly represent the operational inspection constraint.

The final selection objective therefore focused on C4 ranking under a fixed fleet-review budget. The principal measures became the number of C4 vehicles found in the Top 5% of the ranked fleet, Precision at 5%, Recall at 5%, Average Precision, and ROC–AUC. This change in evaluation objective was important: a model could sacrifice some overall five-class performance while still providing substantially greater value by recovering more imminent C4 vehicles within the available inspection capacity.

6.2 Baseline model families

The first experiments compared multinomial logistic regression, Random Forest, and five-class XGBoost. Logistic regression provided a transparent reference model but could not adequately represent nonlinear interactions between vehicle specifications, accumulated operating exposure, and recent changes. Random Forest improved nonlinear modelling but remained weak on the rare maintenance stages and produced less effective C4 rankings than gradient boosting. Five-class XGBoost became the strongest general-purpose baseline because it handled nonlinearities, missing values, and heterogeneous feature scales more effectively.

Despite overall accuracy of approximately 0.95–0.97, the early five-class models showed weak minority-stage performance. The original baseline placed only four of the 60 C4 vehicles in the official Test Top 5%. This result demonstrated that high classification accuracy did not imply useful identification of urgent vehicles. It motivated the separation of two modelling tasks:

1. a specialist binary ranker for C4 inspection prioritisation; and
2. a five-class model for stage communication and cumulative horizon probabilities.

This separation allowed the urgent C4 objective to be optimised directly without removing the more interpretable C0–C4 stage representation from the system.

6.3 Feature-representation experiments

Multiple feature collections were compared to determine how cumulative and temporal information should be represented. The main variants included:

- exposure-normalised rates derived from all cumulative variables;
- rates combined with changes from earlier readouts;
- short-history changes using recent lags;
- longer-history changes and historical summaries;
- recent rates combined with recent-to-lifetime ratios;
- raw latest-state cumulative values;
- removal of raw time-related variables and lifetime exposure features;
- fixed operational-time windows; and
- within-fleet ranks of selected rates and ratios.

The latest-state baseline preserved the current accumulated values but was strongly influenced by vehicle age and total operating exposure. Two vehicles could therefore appear different simply because

one had operated for longer. Normalising counters by accumulated operating time reduced this effect and provided more meaningful cross-vehicle comparisons.

Histogram variables required a separate transformation. Individual histogram bins were divided by their group total to represent the distribution of activity rather than its cumulative magnitude. Changes in histogram composition were then calculated over the previous one, three, and five readouts. These features captured whether a vehicle's recent operating pattern had shifted, while the recent-to-lifetime ratios compared short-term behaviour with the vehicle's own historical baseline.

Short-history features were consistently more useful than very long lags. Changes over one, three, and five readouts captured recent deterioration without requiring every vehicle to have a long and regularly sampled history. Longer lags reduced the number of usable observations, increased missingness, and did not provide a sufficiently stable improvement. Fixed operational-time windows were also investigated to reduce sensitivity to irregular sampling, but they did not outperform the simpler readout-based short-history representation.

Removing raw `time_step` reduced the risk that the model would treat vehicle age as a direct proxy for maintenance proximity. Completely removing lifetime-rate information, however, discarded useful context. Similarly, converting features to within-fleet ranks slightly improved some development results but reduced performance on the official Test data. Fleet-rank normalisation removed magnitude information and made the representation depend on the composition of the fleet being scored. It was therefore not adopted as the main feature transformation.

The final feature contract retained all five informative feature blocks: exposure-normalised rates, histogram proportions, recent histogram-mix changes, recent-to-lifetime ratios, and one-hot encoded vehicle specifications. This produced 534 model features. The experiment showed that reducing time exposure effects did not require removing all historical information; the stronger solution was to express recent behaviour relative to both operating exposure and the vehicle's own lifetime pattern.

6.4 Class imbalance, sampling, and weighting

C4 is extremely rare, so a broad set of imbalance treatments was evaluated. These included random oversampling, negative-class downsampling, SMOTE, class-weight exponents, dense sampling of C4 histories, Balanced Random Forest, and Easy Ensemble methods.

Aggressive inverse-frequency weighting increased the influence of C4 examples but also generated many false positives. The model learned decision boundaries that performed well on selected development samples but did not transfer reliably to unseen vehicles. Random oversampling showed a similar limitation: duplicating rare observations increased their frequency without adding new vehicle-level information.

SMOTE did not provide a stable improvement. Synthetic interpolation is difficult to justify for the engineered vehicle features because the feature space contains ratios, sparse changes, missing historical values, histogram proportions, and one-hot specification indicators. Interpolated points may be mathematically valid while not corresponding to a realistic vehicle history. Consequently, improved development scores did not translate into a sufficiently strong Test ranking.

Negative downsampling reduced training time and increased the relative importance of C4, but aggressive downsampling removed useful variation from the large C0 population. This weakened the model's ability to distinguish genuine high-risk patterns from difficult C0 cases. Balanced Random Forest and Easy Ensemble provided additional imbalance-aware baselines but did not exceed the boosted-tree rankers.

Moderate weighting was more reliable than either no control or full inverse-frequency weighting during the selected specialist-model fitting. The final C4 models used a weight exponent of 0.75, which increased the contribution of C4 without allowing the minority class to dominate the loss. The broader finding was that imbalance could not be solved by weighting alone. Objective selection, endpoint alignment, realistic feature construction, and vehicle-level evaluation were more important than maximising the numerical weight assigned to C4.

6.5 Calibration and probability experiments

Probability calibration was investigated because the tree models sometimes produced overconfident or differently scaled outputs. Validation-fitted calibration and alternative decision thresholds were evaluated for the five-class and binary predictions. Calibration improved the interpretation of some probability values but did not produce a large improvement in the fixed Top-5% C4 queue.

This result follows from the distinction between probability estimation and ranking. A monotonic calibration transformation can change the numerical value of a probability without changing the ordering

of vehicles. It may therefore improve probability interpretation while leaving Precision at 5%, Recall at 5%, and the number of C4 vehicles recovered unchanged. Calibration was also sensitive to the prevalence difference between development and official evaluation data.

A second calibration issue arose when combining XGBoost and LightGBM. Their raw probabilities were not directly comparable because the models produced different score distributions. Direct probability averaging allowed the model with the more extreme probability scale to dominate the ensemble. This led to the adoption of percentile-rank averaging: each model first ranked the fleet, after which the two ranks were averaged and ranked again. This preserved complementary ordering information without assuming that the raw probabilities were calibrated to the same scale.

For user-facing risk communication, the system retained the coherent five-class probability vector rather than presenting the binary ensemble score as a calibrated event probability. Consequently, `risk6_score` stores $P(C4)$ from the five-class model, while `risk6_percentile` stores the specialist ensemble rank used for prioritisation.

6.6 Specialist C4 ranking and ensemble experiments

Once the C4 objective was separated from five-class classification, binary XGBoost and LightGBM models were trained specifically to distinguish C4 from non-C4 vehicles. The experiments varied tree depth, number of estimators, learning rate, minimum child constraints, regularisation, feature subsets, class weighting, random seed, and the number of historical C4 observations retained during training.

A recall-oriented XGBoost model using short temporal features substantially improved C4 recovery, finding 14 of the 60 Test C4 vehicles in the Top 5%, compared with four for the original baseline. This confirmed that specialist binary training was more suitable for the urgent-stage objective. LightGBM produced a different ordering of difficult vehicles and therefore provided a useful complementary learner.

Several ensemble methods were examined:

- direct averaging of XGBoost and LightGBM probabilities;
- percentile-rank averaging;
- seed averaging and seed bagging;
- stacked models trained on base-model predictions;
- cascades that applied a second model to a selected risk region;
- depth blending across shallow and deeper trees; and
- a separate C0-versus-C4 veto model.

Percentile-rank averaging was more robust than raw-probability averaging because it avoided scale incompatibility between the two boosting libraries. Stacking and cascades added complexity but did not provide a sufficiently stable gain after vehicle-level evaluation. Seed averaging reduced some variance, although it did not increase the number of C4 vehicles recovered within the final inspection budget.

The C0-versus-C4 veto model was introduced to improve precision by lowering the rank of candidates that strongly resembled C0. It improved some narrow-budget metrics, particularly near the top of the queue, and sometimes increased Average Precision. However, it also removed genuine C4 vehicles from the wider Top-5% queue. Because the operational objective required stronger C4 recovery at the fixed 5% budget, the veto was treated as an informative precision experiment rather than retained in the final architecture.

6.7 Temporal degradation and sequence experiments

A further group of experiments investigated whether maintenance proximity could be modelled as a degradation stage rather than only as a pointwise classification problem. The evaluated approaches included degradation-stage features, two-stage gates, interval-based histories, hard-negative reranking, sequence representations, GRU-based reranking, and learning-to-rank objectives such as LambdaRank.

The motivation was that a vehicle may enter a period in which several variables begin to drift before the maintenance event. A first-stage model could identify this degradation region, after which a second-stage model could estimate whether the vehicle was approaching C4. In practice, the anonymised variables, irregular observation intervals, short histories for some vehicles, and small number of C4 endpoints limited the benefit of more complex sequence models.

Two-stage gates were sensitive to the first-stage threshold: a false negative at the gate prevented the downstream C4 model from recovering the vehicle. Hard-negative reranking improved selected difficult

cases but could overfit the specific development negatives. GRU and interval-sequence models introduced additional parameters without a sufficiently large number of positive vehicle sequences. LambdaRank aligned training more directly with ordering, but it did not produce a stable advantage over the simpler boosted-tree ensemble.

These experiments nevertheless produced an important feature-engineering finding. Recent changes were valuable, but the full sequence did not need to be modelled by a deep recurrent architecture. Short, past-only summaries over the previous one, three, and five readouts captured much of the useful temporal signal while remaining robust to missing and irregular histories.

6.8 Endpoint alignment and final tree tuning

A major improvement resulted from aligning the training unit with production inference. Earlier models sampled multiple observations from a vehicle and were therefore trained on a mixture of early, intermediate, and endpoint conditions. In production, however, the system scores each vehicle once at its latest available readout. This mismatch allowed vehicles with long histories to contribute more training rows and caused the training distribution to differ from the deployment distribution.

The endpoint-aligned experiments retained one latest eligible observation per vehicle. Five feature families and several tree-complexity profiles were then compared under this fixed protocol. The complete 534-feature representation combined with a flexible XGBoost–LightGBM pair produced the strongest operational result. On the official Test split, the final ensemble recovered 20 of the 60 C4 vehicles in the Top 5% of the fleet.

Further tuning examined deeper and shallower trees, depth blending, alternative regularisation, boundary weighting, additional cascades, and seed bagging. Some variants improved Average Precision, ROC–AUC, or performance at a narrower inspection budget, but none produced a better overall trade-off than the selected architecture. In particular, no alternative exceeded the final model's 20 C4 detections in the Top-5% queue while also maintaining the stronger overall ranking metrics.

Endpoint alignment therefore provided more value than increasing model complexity. The most important change was not a new learner but a closer match between the training sample and the actual decision made by the deployed system.

Table 6: Summary of predictive-modelling experiments and resulting design decisions.

Experimental dimension	Variables examined	Main finding	Final decision
Model family	Logistic regression, Random Forest, XGBoost, LightGBM, balanced ensembles	Gradient-boosted trees provided the strongest nonlinear minority-stage ranking	Use XGBoost and LightGBM
Prediction objective	Five-class accuracy, Macro F1, horizon AP, binary C4 ranking	Overall accuracy concealed weak C4 identification	Optimise fixed-budget C4 ranking separately
Feature representation	Raw cumulative values, rates, proportions, short and long changes, fixed windows, fleet ranks	Exposure-normalised features and short changes transferred most reliably	Retain the complete 534-feature representation
Temporal history	Latest state, multiple sampled histories, lags of 1/3/5, long lags, full sequences	Short histories captured useful deterioration with less missingness and complexity	Use past-only lags of 1, 3, and 5 readouts
Imbalance treatment	Random oversampling, SMOTE, negative downsampling, balanced ensembles, weight exponents	Synthetic and aggressive balancing increased false positives or overfit development data	Use moderate C4 weighting rather than synthetic sampling
Calibration	Validation-fitted calibration, alternative thresholds, raw-probability averaging	Calibration changed probability interpretation more than vehicle ordering	Keep coherent five-class probabilities and rank binary models separately
Ensemble method	Probability averaging, rank averaging, stacking, cascades, seed bagging, veto models	Rank averaging preserved complementary orderings without requiring equal calibration	Use XGBoost–LightGBM percentile-rank averaging
Sequence modelling	Degradation gates, interval sequences, hard-negative reranking, GRU, LambdaRank	Added complexity did not deliver a stable vehicle-level improvement	Prefer engineered temporal summaries with boosted trees
Training unit	Multiple observations per vehicle versus one endpoint per vehicle	Endpoint alignment produced the largest stable improvement	Train and evaluate the specialist C4 ranker using one latest eligible observation per vehicle.
Final tuning	Tree depth, regularisation, depth blending, boundary weighting, seeds, budget cascades	Some variants improved individual metrics but not the complete Top-5% trade-off	Retain the selected flexible 534-feature ensemble

6.9 Key findings and model-selection rationale

The experimental programme produced several findings that determined the final architecture.

First, the evaluation objective was more important than overall accuracy. A model selected for five-class accuracy could perform well numerically while failing to identify the urgent C4 vehicles. Fixed-budget ranking metrics were therefore required to represent the actual fleet decision.

Second, class imbalance could not be solved through aggressive weighting or synthetic oversampling alone. Moderate weighting was useful, but realistic vehicle-level sampling and endpoint alignment produced more reliable gains.

Third, cumulative operational variables required transformation rather than simple inclusion or removal. Exposure-normalised rates, within-histogram proportions, recent changes, and recent-to-lifetime ratios reduced the influence of vehicle age while retaining meaningful operating context.

Fourth, recent temporal behaviour was important, but complex sequence models were not necessary. Past-only changes over one, three, and five readouts provided a stronger balance between signal, missingness, interpretability, and deployment simplicity.

Fifth, calibration and ranking served different purposes. The five-class model was retained to produce coherent stage probabilities, while the specialist binary ensemble was used to rank inspection priority. Percentile-rank averaging allowed XGBoost and LightGBM to be combined without treating their raw probabilities as directly comparable.

Finally, matching training to the production decision was more valuable than further increases in architectural complexity. The selected model was therefore the endpoint-aligned XGBoost–LightGBM C4 rank ensemble using all 534 features, together with a separate five-class XGBoost for stage and horizon probabilities.

7 RAG Evaluation

7.1 Evaluation Setup

The evaluation examined FleetCare AI's ability to retrieve maintenance knowledge, generate natural-language explanations, and access stored vehicle assessments. Document-based experiments were complemented by vehicle-tool and HTTP checks to cover the main assistant workflow.

The knowledge base contained five sources and 274 chunks, comprising 98 maintenance, 168 policy, and eight project chunks. Retrieval used 384-dimensional multilingual MiniLM embeddings and pgvector cosine similarity, with category filtering, a minimum similarity of 0.30, and up to five passages per query. Review-date filtering and Singapore/global eligibility were applied to policy searches. Qwen 3.5 Flash generated responses with thinking disabled, while PostgreSQL-compatible PGlite hosted the local evaluation database.

The question set included eight answerable maintenance, policy, and project questions, together with two boundary cases concerning unavailable OEM repair details and an out-of-domain request. Reference citation identifiers and expected answer points were taken from the project evaluation materials and checked against the knowledge corpus.

7.2 Evaluation Protocol

Three complementary configurations were evaluated. *Controlled retrieval* supplied the known knowledge category to the implemented `KnowledgeSearchTool`. *Fixed-evidence generation* provided the resulting passages directly to Qwen without tool selection, using temperature 0 and a maximum of 1800 output tokens. The integrated assistant used the end-to-end backend orchestration, automatic tool selection, and up to five tool-calling rounds. Reference answers and gold identifiers were excluded from generation inputs.

For an answerable question q, let G_q denote the annotated gold citation set and $R_q^{(5)}$ the first five returned identifiers. Retrieval was measured by

$$P@5_q = \frac{|G_q \cap R_q^{(5)}|}{5}, \qquad R@5_q = \frac{|G_q \cap R_q^{(5)}|}{|G_q|}. \tag{10}$$

Results were macro-averaged over the eight answerable questions. Integrated-assistant retrieval was scored using the first knowledge-tool call, with an omitted call treated as an empty result; subsequent calls remained available for response generation. Boundary questions were assessed through their responses

rather than retrieval precision and recall. These metrics measure matching against the annotated reference passages, which do not exhaust all potentially relevant corpus content.

Response quality was assessed automatically using Qwen across four 0–2 dimensions: relevance, correctness, grounding, and scope, giving a maximum of eight points. The evaluator received the question, generated answer, expected points, and actual retrieved evidence. Scores are same-model rubric assessments rather than answer-accuracy estimates. Execution completion, response latency, and citation behaviour were also recorded.

7.3 Results and Analysis

Both answer-generation configurations completed all ten questions without API failures or observed output-length truncation. Table 7 presents the aggregate results. The completion measure concerns successful execution; response quality is reported separately through the automated rubric.

Table 7: RAG evaluation results. Retrieval metrics use eight answerable questions; response scores and generation times use all ten questions.

Metric	Controlled retrieval	Fixed-evidence generation	Integrated assistant
Execution completion	10/10	10/10	10/10
Precision@5	0.175	–	0.150
Recall@5	0.5313	–	0.4063
Automated score (/8)	–	7.60	5.40
Mean time (s)	0.23	4.98	4.40

Evidence retrieval and answer generation Controlled retrieval recovered at least one annotated passage for six of the eight answerable questions, demonstrating access to relevant maintenance, policy, and project evidence. Fixed-evidence generation achieved a mean automated score of 7.60/8, indicating favourable rubric assessments when retrieved context was supplied directly. The integrated assistant combined automatic tool selection and response generation within a mean of 4.40 seconds per question, returning answers for all ten requests. These observations demonstrate an operational connection between the knowledge base, retrieval tools, and language model.

The integrated assistant's score of 5.40/8 reflects the additional requirements of selecting tools, reformulating queries, and deciding when to retrieve evidence. The two generation configurations use different prompts and settings, so their scores describe the observed workflows rather than isolate the causal effect of tool calling. Retrieval timing includes initial embedding-model loading; generation measurements followed this warm-up.

Vehicle evidence and application integration Three separate synthetic vehicle cases verified access to the latest stored action and rule, handling of a missing assessment, and retrieval of five stored SHAP factors. All three produced the expected tool calls and fixture fields, with a mean automated response score of 7.33/8. Table 8 summarises these checks together with the HTTP interface tests.

The unavailable-OEM case also demonstrated evidence-aware abstention: the integrated assistant acknowledged the absence of the requested repair procedure, part number, and torque specification. The vehicle checks establish retrieval and interface behaviour using known test records; they do not measure real-vehicle decision accuracy. The main document experiment invoked the backend orchestrator directly, while HTTP behaviour was checked separately.

Interpretation and refinement The observed refinements concern citation coverage, precise wording of operating horizons, evidence-to-answer consistency, and out-of-domain handling. Two of the eight answerable integrated responses included document citation identifiers, while some responses introduced unsupported details or did not cite the retrieved evidence consistently.

The small question set, single execution per case, and same-model automated evaluator limit generalisation; the scores should therefore be read as diagnostic results for this local experiment.

Table 8: Vehicle-tool and HTTP integration checks. Vehicle cases use synthetic records and are reported separately from the document-question experiment.

Check	Observed result
Latest vehicle assessment	Expected stored action and rule retrieved.
Missing vehicle assessment	Missing-record condition returned by the tool.
SHAP evidence	Five stored feature contributions retrieved.
Health endpoint	HTTP 200 with the expected health response.
Request validation	HTTP 422 for a request without a message.
Chat endpoint	HTTP 200 with a generated vehicle-assessment response.

8 Conclusion

FleetCare AI demonstrates how predictive modelling, temporal reasoning, explainability, and knowledge retrieval can be integrated into an end-to-end predictive-maintenance decision-support system. Rather than treating prediction as an isolated classification task, the system connects vehicle data, fleet-relative risk ranking, explicit operational rules, stored evidence, and a user-facing maintenance workflow.

The final predictive architecture separates inspection prioritisation from vehicle-level risk communication. An endpoint-aligned XGBoost–LightGBM ensemble ranks vehicles by near-term C4 risk, while a separate five-class XGBoost model provides the displayed maintenance stage and cumulative Risk6, Risk12, Risk24, and Risk48 probabilities. On the official Test split, the final ranker identified 20 of the 60 C4 vehicles within the Top 5% of the fleet, compared with four for the original baseline. Precision at this inspection capacity increased from 1.58% to 7.91%, while ROC–AUC increased from 0.6722 to 0.7907. These results show that aligning the modelling objective with a limited inspection budget was more useful than optimising conventional five-class accuracy alone.

The ordered rule engine converts model rankings and recent temporal evidence into traceable actions. It preserves the model-defined priority queue, handles persistent or deteriorating risk, routes volatile or disagreeing predictions to manual review, and prevents vehicles with insufficient history from receiving overconfident recommendations. Although the additional TR2B rescue condition did not outperform direct model ranking at equal capacity, it provides an explicit and auditable mechanism for escalating vehicles with jointly high near-term risk and deterioration.

The completed application preserves both model and decision explanations. SHAP identifies the anonymised features that most influenced the C4 XGBoost component, while the rule trace records why a particular operational action was assigned. PostgreSQL maintains the assessment history, the React dashboard presents fleet and vehicle evidence, and the Qwen-based assistant retrieves stored assessments and citation-bearing maintenance knowledge through controlled tools. Evaluation confirmed that these components operate together, while also identifying remaining limitations in citation coverage, probability interpretation, and generalisation from a small benchmark and evaluation set.

FleetCare AI should therefore be interpreted as a human-in-the-loop maintenance decision-support system rather than an autonomous diagnostic system. Its main contribution is a transparent reasoning chain from historical vehicle data to inspection priority, operational action, supporting evidence, and human review. With meaningful non-anonymised features, organisation-specific maintenance policies, prospective fleet validation, and continuous monitoring, the same architecture could support practical maintenance planning in a live commercial fleet environment.

9 Future Work

Future development should focus on adapting the completed system from an anonymised benchmark setting to a continuously operated fleet environment. Four directions are particularly important.

1. **Use physically meaningful operational features.** The current dataset anonymises both operational variables and vehicle specifications. Consequently, SHAP can identify influential feature IDs and the direction of their contribution, but it cannot translate them into component-specific inspection advice. In a production environment, the model should be retrained using documented

telemetry, diagnostic codes, component information, usage conditions, and maintenance history. This would allow explanations to be connected to meaningful operational signals and support more specific recommendations for technicians.

2. **Align the rule engine with real operational policy.** The current rule engine demonstrates how model rankings, temporal trends, history sufficiency, and uncertainty can be converted into transparent actions. Its thresholds and actions should next be reviewed with fleet managers and maintenance specialists. Once meaningful features are available, rules could incorporate component criticality, vehicle duty, workshop capacity, parts availability, safety requirements, and the cost of missed or unnecessary inspections. Rule versions should be recorded and evaluated prospectively so that policy changes remain auditable.
3. **Extend RAG with internal company knowledge.** The retrieval-augmented assistant can be strengthened using authorised internal sources such as maintenance policies, service manuals, inspection procedures, historical work orders, and escalation guidelines. Access control and document-version tracking would be required to ensure that each response uses current and permitted information. The assistant should continue to cite its sources and preserve the deterministic action produced by the rule engine rather than generating an unsupported alternative.
4. **Establish continuous learning and monitoring.** New operational readouts, inspection outcomes, confirmed maintenance events, and technician feedback should be collected as the fleet operates. These data can support scheduled model retraining, probability recalibration, and refinement of both model and rule thresholds. A production process should also monitor input drift, ranking performance, action volumes, and C4 capture at fixed inspection capacities. Updated models should be evaluated on a later untouched time period and deployed only after comparison with the currently approved version.

Together, these extensions would move FleetCare AI from a benchmark-based decision-support system toward a domain-specific platform whose predictions, rules, explanations, and knowledge base evolve with the fleet's operating and maintenance experience.

10 Appendix

10.1 Installation and User Guide

FleetCare AI is implemented using React, FastAPI, PostgreSQL 16 with pgvector, and a Qwen-powered AI assistant. The application requires a configured Python environment, Node.js, PostgreSQL, and the corresponding database and knowledge resources. Detailed installation procedures, environment configuration, database setup, and instructions for operating the Fleet Dashboard, Vehicle Analysis, and AI Assistant are documented in Section 5 of the project's GitHub README.

10.2 Mapped System Functionalities

FleetCare AI integrates knowledge, techniques, and practical skills from Machine Reasoning (MR), Reasoning Systems (RS), and Cognitive Systems (CGS). The following tables map the implemented functionalities to these courses and identify the skills demonstrated in developing the system. The mappings indicate the main connection of each functionality; several components also contribute across courses.

Machine Reasoning (MR) The project applies data-driven knowledge discovery and explicit rule-based reasoning to transform vehicle histories into explainable maintenance recommendations. Table 9 summarises these connections.

The predictive models and their evaluation are described in Sections 5.3 and 6; the rule representation and execution order are detailed in Section 5.4.

Table 9: Mapping of FleetCare AI functionalities to Machine Reasoning (MR).

System functionality	Knowledge and techniques applied	Skills demonstrated
Predictive knowledge discovery	Supervised learning and temporal feature engineering. A 534-feature representation supports an XGBoost–LightGBM C4 ranker and a separate five-class prediction model.	Prepare longitudinal data, construct past-only features, handle class imbalance, and evaluate rare-event ranking using Average Precision and C4 capture at fixed budgets.
Rule-based maintenance decisions	Ordered IF–THEN rules combine fleet-relative rankings with temporal evidence. The first matching rule assigns inspection, monitoring, review, or data-collection actions.	Represent decision policies as executable rules, define precedence, and retain the matched rule, reason, and evidence for traceable recommendations.
Incomplete and unstable evidence handling	Persistence, deterioration, volatility, and model disagreement describe recent prediction behaviour. Review and data-collection rules complement the model-led inspection rules.	Assess history sufficiency, combine current and historical evidence, and implement explicit review routes while preserving rule precedence.

Reasoning Systems (RS) The RS contribution is reflected in decomposing the maintenance problem into cooperating intelligent subsystems and integrating their outputs under operational constraints. Table 10 presents this mapping.

Table 10: Mapping of FleetCare AI functionalities to Reasoning Systems (RS).

System functionality	Knowledge and techniques applied	Skills demonstrated
Hybrid reasoning architecture	Learned predictions become inputs to temporal assessment and symbolic action rules. PostgreSQL stores the resulting assessments for retrieval by the dashboard and assistant.	Decompose a business problem, assign subsystem responsibilities, and integrate prediction, reasoning, persistence, and user interaction through consistent data interfaces.
Inspection prioritisation under constraints	Fleet-relative ranking supports a limited inspection budget. TR1/TR2 select the initial Top-5% queue; TR2B can add cases meeting an explicit deterioration condition.	Translate operational constraints into decision logic, compare C4 capture at equal inspection budgets, and distinguish prioritisation from search-based resource optimisation.

The architecture and inference workflow are described in Sections 5.1 and 5.5. This implementation demonstrates hybrid-system design and integration; it does not implement an informed-search or evolutionary-computing optimiser for workshop scheduling.

Cognitive Systems (CGS) The cognitive layer makes structured assessments and documented knowledge accessible through natural-language interaction and visual explanations. Table 11 summarises the relevant techniques and skills.

The assistant and user interfaces are described in Section 5.7. The assistant retrieves and explains stored assessments; its tools do not change rule-engine actions. SHAP contributions indicate predictive associations with anonymised features, rather than confirmed mechanical failure causes.

Table 11: Mapping of FleetCare AI functionalities to Cognitive Systems (CGS).

System functionality	Knowledge and techniques applied	Skills demonstrated
Knowledge representation and retrieval	Document chunks retain source metadata and citation identifiers. Multilingual embeddings and pgvector similarity search retrieve evidence, with category, jurisdiction, and review-date filters.	Represent textual knowledge for retrieval, combine semantic matching with metadata constraints, and preserve source provenance for cited responses.
Natural-language assistant	Qwen Function Calling selects backend tools for vehicle assessments or maintenance knowledge. Retrieved evidence is returned to the model for response generation.	Define tool interfaces, connect user questions to structured and textual evidence, and integrate an LLM with backend-controlled retrieval.
Human-centred explanations	The dashboard presents fleet priorities, vehicle histories, SHAP contributions, and rule evidence. SHAP explains the C4 XGBoost component; rule evidence explains action selection.	Design interpretable interfaces, distinguish model contributions from decision rules, and support human review of recommendations.

Alignment with the practice-module requirements The project integrates three of the four technique groups specified in the practice-module briefing: decision automation, knowledge discovery, and cognitive techniques. Table 12 summarises the implemented coverage.

Table 12: Alignment with the practice-module technique groups.

Technique group	Coverage	System implementation
Decision automation	Demonstrated	Ordered business rules translate model outputs and temporal evidence into traceable maintenance actions.
Business resource optimisation	Not claimed	Inspection ranking supports prioritisation, but no informed-search or evolutionary-computing optimiser is implemented.
Knowledge discovery and data mining	Demonstrated	Temporal feature engineering, supervised prediction, and ensemble ranking extract repair-proximity patterns from vehicle histories.
Cognitive techniques and natural-language interaction	Demonstrated	A document knowledge base, semantic retrieval, and a tool-assisted chatbot provide access to maintenance references and vehicle assessments.

The React interface, FastAPI services, and PostgreSQL records connect these techniques into one decision-support workflow. Fleet summaries, historical charts, SHAP factors, and rule evidence provide results interpretation and visualisation. Reuse of established modelling libraries, pgvector, and the Qwen API also follows the briefing's encouragement to integrate existing software components. Final inspection and repair decisions remain with qualified personnel.

10.3 Proposal Report

The original FleetCare AI project proposal is available as a separate 19-page PDF. It documents the initial business problem, market research, project objectives and scope, data preparation strategy, and proposed predictive modelling, temporal reasoning, explainability, dashboard, and RAG components. The proposal is retained as a record of the project's initial direction; the final implementation, experimental results, and limitations are presented in the main body of this report.
View proposal: FleetCare AI Project Proposal (PDF)
Download proposal: Download AIS_Proposal.pdf

References

1. Brunheroto, P.H., Pepino, A.L.G., Deschamps, F., Loures, E.d.F.R.: Data analytics in fleet operations: A systematic literature review and workflow proposal. Procedia CIRP **107**, 1192–1197 (2022). https://doi.org/10.1016/j.procir.2022.05.130 4
2. Carvalho, T.P., Soares, F.A.A.M.N., Vita, R., Francisco, R.d.P., Basto, J.P., Alcalá, S.G.S.: A systematic literature review of machine learning methods applied to predictive maintenance. Computers & Industrial Engineering **137**, 106024 (2019). https://doi.org/10.1016/j.cie.2019.106024 4
3. Geotab: Fleet maintenance solutions and software. https://www.geotab.com/fleet-management-solutions/fleet-maintenance/, accessed: 13 September 2026 4
4. IBM: IBM Maximo Application Suite. https://www.ibm.com/products/maximo, accessed: 13 September 2026 4
5. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. In: Advances in Neural Information Processing Systems. vol. 30, pp. 4765–4774 (2017) 4
6. Samsara: Connected maintenance. https://www.samsara.com/products/telematics/fleet-maintenance, accessed: 13 September 2026 4
7. Scania: Digital services: Making your business smarter. https://www.scania.com/sg/en/home/services/data-driven-services.html, accessed: 13 September 2026 4
8. Theissler, A., Pérez-Velázquez, J., Kettelgerdes, M., Elger, G.: Predictive maintenance enabled by machine learning: Use cases and challenges in the automotive industry. Reliability Engineering & System Safety **215**, 107864 (2021). https://doi.org/10.1016/j.ress.2021.107864 4